Stage 1 Class Diagram

Basic operation - regular customers, base menu, three staff roles and the complete restaurant lifecycle

Exact source snapshot: stage-1 @ 9133246 - all declared attributes and operations

+ public # protected - private ~ package/default underlined = static italic = abstract hollow/filled diamond = aggregation/composition hollow triangle = generalisation dashed arrow = selected creation/use dependency

default package

<<application>>
Main

+ Main() {implicit}
+ main(args: String[]): void

boundaries

<<boundary>>
ControlUI.MenuOption
(static, final)

- id: String {readOnly}
- label: String {readOnly}

+ MenuOption(id: String, label: String)
+ getId(): String
+ getLabel(): String
+ toString(): String

<<boundary>>
ControlUI
(final)

- DIVIDER: String {readOnly}
- inputReader: InputReader {readOnly}
- output: PrintStream {readOnly}

+ ControlUI()
+ ControlUI(inputReader: InputReader)
+ ControlUI(inputReader: InputReader, output: PrintStream)
+ showWelcome(seed: long): void
+ showTurnStart(turnNumber: int, maximumTurns: int, revenueCents: int averagePaidSatisfaction: Double, waitingQueue: List<String> occupiedTables: List<String>, customerStates: List<String>): void
+ askWaiterTask(availableOptions: List<MenuOption>): MenuOption
+ askChefTask(availableOptions: List<MenuOption>): MenuOption
+ askTask(staffName: String, availableOptions: List<MenuOption>): MenuOption
+ showArrival(customerDescription: String): void
+ showOrderPlaced(customerDescription: String, menuItemName: String lockedPriceCents: int, preparationUnits: int): void
+ showEvent(message: String): void
+ showActionSucceeded(message: String): void
+ showActionRejected(reason: String): void
+ showTurnResults(turnsUsed: int, servedCount: int, paidCount: int unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
+ showFinalResult(victory: boolean, turnsUsed: int, servedCount: int, paidCount: int unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
+ formatMoney(amountInCents: int): String
- showResults(heading: String, turnsUsed: int, servedCount: int, paidCount: int unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
- printList(heading: String, items: List<String>, emptyMessage: String): void
- formatAverageSatisfaction(averagePaidSatisfaction: Double): String
- requireList(list: List<?>, name: String): void
- requireText(text: String, name: String): String

<<boundary>>
InputReader
(final)

- scanner: Scanner {readOnly}
- output: PrintStream {readOnly}

+ InputReader()
+ InputReader(scanner: Scanner, output: PrintStream)
+ readInt(prompt: String): int
+ readInt(prompt: String, minimum: int, maximum: int): int
+ readChoice(prompt: String, numberOfOptions: int): int
+ readLong(prompt: String): long
+ readYesNo(prompt: String): boolean
+ readNonBlank(prompt: String): String
- readLine(prompt: String): String

control

<<control>>
TurnController

- config: GameConfig {readOnly}
- restaurant: Restaurant {readOnly}
- ui: ControlUI {readOnly}
- rng: Random {readOnly}
- currentTurn: int = 0

+ TurnController(config: GameConfig, ui: ControlUI)
+ run(): void
- playTurn(): void
- resetStaff(): void
- handleArrivals(): void
- buildWaiterOptions(): Map<MenuOption, Supplier<ActionResult>>
- waiterPhase(): void
- buildChefOptions(): Map<MenuOption, Supplier<ActionResult>>
- chefPhase(): void
- eatingAndPayment(): void
- endOfTurnDecay(): void
- waitingQueueLine(): List<String>
- occupiedTableLine(): List<String>
- customerStateLine(): List<String>
- averageOrNull(): Double
- checkVictory(): boolean

<<control>>
ActionResult

- success: boolean {readOnly}
- message: String {readOnly}

+ ActionResult(success: boolean, message: String)
+ isSuccess(): boolean
+ getMessage(): String

<<control>>
Restaurant

- waitingQueue: Queue<Customer> = new LinkedList<>() {readOnly}
- activeOrders: List<Order> = new ArrayList<>() {readOnly}
- tables: List<Table> = new ArrayList<>() {readOnly}
- menu: List<MenuItem> = new ArrayList<>() {readOnly}
- waiter: Waiter {readOnly}
- chef: Chef {readOnly}
- cashier: Cashier {readOnly}
- revenueCents: int = 0
- servedCount: int = 0
- unhappyDepartures: int = 0
- paidSatisfactions: List<Integer> = new ArrayList<>() {readOnly}

+ Restaurant(numTables: int)
+ getFreeTables(): List<Table>
+ getTables(): List<Table>
+ getMenuItems(): List<MenuItem>
+ getWaitingQueue(): Collection<Customer>
+ getSeatedWithoutOrder(): List<Customer>
+ getReadyOrders(): List<Order>
+ getOrdersInPrep(): List<Order>
+ getEatingCustomers(): List<Customer>
+ getReadyToPayCustomers(): List<Customer>
+ getCustomersLosingSatisfaction(): List<Customer>
+ getOrderFor(c: Customer): Order
+ getTableOf(c: Customer): Table
+ getRevenueCents(): int
+ getServedCount(): int
+ getUnhappyDepartures(): int
+ getPaidCount(): int
+ getAverageSatisfaction(): OptionalDouble
+ addToQueue(c: Customer): void
+ getWaiter(): Waiter
+ getCashier(): Cashier
+ getChef(): Chef
+ getAllStaff(): List<Staff>
+ seatCustomer(c: Customer, t: Table): ActionResult
+ takeOrder(c: Customer, mg: Random, currentTurn: int): ActionResult
+ advanceOrder(o: Order, currentTurn: int): ActionResult
+ serveOrder(o: Order, currentTurn: int): ActionResult
+ processPayment(c: Customer, currentTurn: int): String
+ processEndOfTurn(c: Customer): boolean

<<control>>
GameConfig

+ MAX_TURNS: int = 18 {readOnly}
+ NUM_TABLES: int = 2 {readOnly}
+ TARGET_PAID: int = 4 {readOnly}
+ TARGET_REVENUE: int = 4500 {readOnly}
+ TARGET_AVG_SATISFACTION: int = 60 {readOnly}
- TURNS: Integer[] = {1, 4, 7, 10, 13, 16} {readOnly}
- seed: long {readOnly}

+ GameConfig(seed: long)
+ returnCustomerThisTurn(turn: int): Customer
+ getSeed(): long

util

<<utility>>
MoneyFormatter
(final)

- MoneyFormatter()
+ format(amountInCents: int): String

entities

<<entity>>
Staff
(abstract)

- name: String
- actedThisTurn: boolean = false

+ Staff(name: String)
+ getName(): String
+ canAct(): boolean
+ markActed(): void
+ resetForNewTurn(): void

<<entity>>
Chef
(final)

+ Chef()
+ Chef(name: String)
+ prepareOneUnit(order: Order, turn: int): boolean

<<entity>>
Cashier
(final)

+ Cashier()
+ Cashier(name: String)
+ collectPayment(customer: Customer, turn: int): int

<<entity>>
Waiter
(final)

+ Waiter()
+ Waiter(name: String)
+ seatCustomer(customer: Customer, table: Table): void
+ takeOrder(customer: Customer, item: MenuItem, orderId: int, turn: int): Order
+ takeOrder(customer: Customer, item: MenuItem, turn: int): Order
+ serve(customer: Customer, turn: int): void

<<entity>>
Customer
(abstract)

- nextId: int = 1
- customerId: int {readOnly}
- arrivalTurn: int {readOnly}
- satisfaction: int
- status: CustomerStatus
- order: Order
- table: Table
- servedTurn: int
- readyToPayTurn: int
- satisfactionAtPayment: int

Customer(arrivalTurn: int)
+ *getSatisfactionLoss(): int*
+ applyEndOfTurn(): boolean
+ decreaseSatisfaction(): void
- abandon(): void
+ markSeated(assignedTable: Table): void
+ chooseMenuItem(menu: List<MenuItem>, rng: Random): MenuItem
+ attachOrder(newOrder: Order): void
+ markServed(turn: int): void
+ finishEating(currentTurn: int): boolean
+ markPaid(currentTurn: int): int
+ releaseTable(table: Table): void
+ getCustomerId(): int
+ getArrivalTurn(): int
+ getSatisfaction(): int
+ getStatus(): CustomerStatus
+ getOrder(): Order
+ getTable(): Table
+ getServedTurn(): int
+ getReadyToPayTurn(): int
+ getSatisfactionAtPayment(): int

<<entity>>
RegularCustomer
(final)

- SATISFACTION_LOSS_PER_TURN: int = 8 {readOnly}

+ RegularCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int

<<entity>>
Table
(final)

- tableNumber: int {readOnly}
- customer: Customer

+ Table(tableNumber: int)
+ getTableNumber(): int
+ getCustomer(): Customer
+ isOccupied(): boolean
+ seatCustomer(customer: Customer): void
+ freeTable(): void
+ toString(): String

<<entity>>
Order
(final)

- generatedIds: int = 1
- orderId: int {readOnly}
- customer: Customer {readOnly}
- menuItem: MenuItem {readOnly}
- priceCents: int {readOnly}
- requiredPreparationUnits: int {readOnly}
- createdTurn: int {readOnly}
- completedPreparationUnits: int
- readyTurn: int = -1
- servedTurn: int = -1
- paidTurn: int = -1
- status: OrderStatus

+ Order(orderId: int, customer: Customer, menuItem: MenuItem, createdTurn: int)
+ Order(customer: Customer, menuItem: MenuItem, createdTurn: int)
~ nextGeneratedId(): int
+ advancePreparation(turn: int): boolean
~ markServed(turn: int): void
~ markPaid(turn: int): void
+ cancel(): void
+ getOrderId(): int
+ getCustomer(): Customer
+ getMenuItem(): MenuItem
+ getPriceCents(): int
+ getRequiredPreparationUnits(): int
+ getCompletedPreparationUnits(): int
+ getCreatedTurn(): int
+ getReadyTurn(): int
+ getServedTurn(): int
+ getPaidTurn(): int
+ getStatus(): OrderStatus

<<enumeration>>
CustomerStatus

WAITING
SEATED
SERVED
READY_TO_PAY
PAID
LEFT

<<enumeration>>
OrderStatus

ORDERED
READY
SERVED
PAID
CANCELLED

<<entity>>
MenuItem
(final)

+ SALAD: MenuItem = new MenuItem("Salad", 900, 1) {readOnly}
+ BURGER: MenuItem = new MenuItem("Burger", 1500, 1) {readOnly}
+ PASTA: MenuItem = new MenuItem("Pasta", 1800, 2) {readOnly}
- STAGE_ONE_MENU: List<MenuItem> = Arrays.asList(SALAD, BURGER, PASTA) {readOnly}
- name: String {readOnly}
- priceCents: int {readOnly}
- preparationUnits: int {readOnly}

+ MenuItem(name: String, priceCents: int, preparationUnits: int)
+ stageOneMenu(): List<MenuItem>
+ getName(): String
+ getPriceCents(): int
+ getPrice(): int
+ getPreparationUnits(): int
+ getPrepUnits(): int
+ toString(): String

<<create>>
<<use>>
<<nest>>
ui
config
restaurant
chef
cashier
waiter
waitingQueue {ordered}
table / customer
order / customer
status
activeOrders {ordered}
tables {ordered}
menu {ordered}
menuItem
inputReader

Stage 2 Class Diagram

Customer behaviour - adds VIPCustomer and CriticCustomer through polymorphic specialisation

Exact source snapshot: stage-2 @ eae1e2c - all declared attributes and operations

+ public # protected - private ~ package/default underlined = static italic = abstract hollow/filled diamond = aggregation/composition hollow triangle = generalisation dashed arrow = selected creation/use dependency

default package

<<application>>
Main

+ Main() {implicit}
+ main(args: String[]): void

boundaries

<<boundary>>
ControlUI.MenuOption
{static, final}

- id: String {readOnly}
- label: String {readOnly}

+ MenuOption(id: String, label: String)
+ getId(): String
+ getLabel(): String
+ toString(): String

<<boundary>>
ControlUI
{final}

- DIVIDER: String {readOnly}
- inputReader: InputReader {readOnly}
- output: PrintStream {readOnly}

+ ControlUI()
+ ControlUI(inputReader: InputReader)
+ ControlUI(inputReader: InputReader, output: PrintStream)
+ showWelcome(seed: long): void
+ formatCustomerLabel(customerId: int, customerType: String): String
+ showTurnStart(turnNumber: int, maximumTurns: int, revenueCents: int, averagePaidSatisfaction: Double, waitingQueue: List<String>, occupiedTables: List<String>, customerStates: List<String>): void
+ askWaiterTask(availableOptions: List<MenuOption>): MenuOption
+ askChefTask(availableOptions: List<MenuZ.ntion>): MenuOption
+ askTask(staffName: String, availableOptions: List<MenuOption>): MenuOption
+ showArrival(customerDescription: String): void
+ showArrival(customerId: int, customerType: String): void
+ showOrderPlaced(customerDescription: String, menuItemName: String, lockedPriceCents: int, preparationUnits: int): void
+ showOrderPlaced(customerDescription: String, menuItemName: String, basePriceCents: int, chargedPriceCents: int, preparationUnits: int): void
+ showCustomerBehaviour(customerDescription: String, behaviourMessage: String): void
+ showPayment(customerDescription: String, amountPaidCents: int): void
+ showEvent(message: String): void
+ showActionSucceeded(message: String): void
+ showActionRejected(reason: String): void
+ showTurnResults(turnsUsed: int, servedCount: int, paidCount: int unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
+ showFinalResult(victory: boolean, turnsUsed: int, servedCount: int, paidCount: int unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
+ formatMoney(amountInCents: int): String
- showResults(heading: String, turnsUsed: int, servedCount: int, paidCount: int unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
- printList(heading: String, items: List<String>, emptyMessage: String): void
- formatAverageSatisfaction(averagePaidSatisfaction: Double): String
- requireNotList(list: List<?>, name: String): void
- requireText(text: String, name: String): String

<<boundary>>
InputReader
{final}

- scanner: Scanner {readOnly}
- output: PrintStream {readOnly}

+ InputReader()
+ InputReader(scanner: Scanner, output: PrintStream)
+ readInt(prompt: String): int
+ readInt(prompt: String, minimum: int, maximum: int): int
+ readChoice(prompt: String, numberOfOptions: int): int
+ readLong(prompt: String): long
+ readYesNo(prompt: String): boolean
+ readNonBlank(prompt: String): String
- readLine(prompt: String): String

entities

<<entity>>
Staff
(abstract)

- name: String
- actedThisTurn: boolean = false

+ Staff(name: String)
+ getName(): String
+ canAct(): boolean
+ markActed(): void
+ resetForNewTurn(): void

<<entity>>
Cashier
{final}

+ Cashier()
+ Cashier(name: String)
+ collectPayment(customer: Customer, turn: int): int

<<entity>>
Waiter
{final}

+ Waiter()
+ Waiter(name: String)
+ seatCustomer(customer: Customer, table: Table): void
+ takeOrder(customer: Customer, item: MenuItem, orderId: int, turn: int): Order
+ takeOrder(customer: Customer, item: MenuItem, turn: int): Order
+ serve(customer: Customer, turn: int): void

<<entity>>
Chef
{final}

+ Chef()
+ Chef(name: String)
+ prepareOneUnit(order: Order, turn: int): boolean

<<entity>>
Customer
(abstract)

- nextId: int = 1
- customerId: int {readOnly}
- arrivalTurn: int {readOnly}
- satisfaction: int
- status: CustomerStatus
- order: Order
- table: Table
- servedTurn: int
- readyToPayTurn: int
- satisfactionAtPayment: int

Customer(arrivalTurn: int)
+ *getSatisfactionLoss(): int*
+ getPriceCents(menuItem: MenuItem): int
+ applyEndOfTurn(): boolean
+ decreaseSatisfaction(): void
- abandon(): void
+ markSeated(assignedTable: Table): void
+ chooseMenuItem(menu: List<MenuItem>, rng: Random): MenuItem
+ attachOrder(newOrder: Order): void
+ markServed(turn: int): void
+ finishEating(currentTurn: int): boolean
+ markPaid(currentTurn: int): int
+ releaseTable(table: Table): void
+ getCustomerId(): int
+ getArrivalTurn(): int
+ getSatisfaction(): int
+ getStatus(): CustomerStatus
+ getOrder(): Order
+ getTable(): Table
+ getServedTurn(): int
+ getReadyToPayTurn(): int
+ getSatisfactionAtPayment(): int
+ *getTypeName(): String*

<<entity>>
CriticCustomer
{final}

- SATISFACTION_LOSS_PER_TURN: int = 12 {readOnly}

+ CriticCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getTypeName(): String
+ chooseMenuItem(menu: List<MenuItem>, rng: Random): MenuItem

<<entity>>
RegularCustomer
{final}

- SATISFACTION_LOSS_PER_TURN: int = 8 {readOnly}

+ RegularCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getTypeName(): String

<<entity>>
VIPCustomer
{final}

- SATISFACTION_LOSS_PER_TURN: int = 5 {readOnly}

+ VIPCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getPriceCents(menuItem: MenuItem): int
+ getTypeName(): String

<<entity>>
Table
{final}

- tableNumber: int {readOnly}
- customer: Customer

+ Table(tableNumber: int)
+ getTableNumber(): int
+ getCustomer(): Customer
+ isOccupied(): boolean
+ seatCustomer(customer: Customer): void
+ freeTable(): void
+ toString(): String

<<entity>>
Order
{final}

- generatedIds: int = 1
- orderId: int {readOnly}
- customer: Customer {readOnly}
- menuItem: MenuItem {readOnly}
- priceCents: int {readOnly}
- requiredPreparationUnits: int {readOnly}
- createdTurn: int {readOnly}
- completedPreparationUnits: int
- readyTurn: int = -1
- servedTurn: int = -1
- paidTurn: int = -1
- status: OrderStatus

+ Order(orderId: int, customer: Customer, menuItem: MenuItem, createdTurn: int)
+ Order(customer: Customer, menuItem: MenuItem, createdTurn: int)
~ nextGeneratedId(): int
+ advancePreparation(turn: int): boolean
~ markServed(turn: int): void
~ markPaid(turn: int): void
+ cancel(): void
+ getOrderId(): int
+ getCustomer(): Customer
+ getMenuItem(): MenuItem
+ getPriceCents(): int
+ getRequiredPreparationUnits(): int
+ getCompletedPreparationUnits(): int
+ getCreatedTurn(): int
+ getReadyTurn(): int
+ getServedTurn(): int
+ getPaidTurn(): int
+ getStatus(): OrderStatus

<<enumeration>>
CustomerStatus

WAITING
SEATED
SERVED
READY_TO_PAY
PAID
LEFT

<<enumeration>>
OrderStatus

ORDERED
READY
SERVED
PAID
CANCELLED

<<entity>>
MenuItem
{final}

+ SALAD: MenuItem = new MenuItem("Salad", 900, 1) {readOnly}
+ BURGER: MenuItem = new MenuItem("Burger", 1500, 1) {readOnly}
+ PASTA: MenuItem = new MenuItem("Pasta", 1800, 2) {readOnly}
- STAGE_ONE_MENU: List<MenuItem> = Arrays.asList(SALAD, BURGER, PASTA) {readOnly}
- name: String {readOnly}
- priceCents: int {readOnly}
- preparationUnits: int {readOnly}

+ MenuItem(name: String, priceCents: int, preparationUnits: int)
+ stageOneMenu(): List<MenuItem>
+ getName(): String
+ getPriceCents(): int
+ getPrice(): int
+ getPreparationUnits(): int
+ getPrepUnits(): int
+ toString(): String

control

<<control>>
TurnController

- config: GameConfig {readOnly}
- restaurant: Restaurant {readOnly}
- ui: ControlUI {readOnly}
- rng: Random {readOnly}
- currentTurn: int = 0

+ TurnController(config: GameConfig, ui: ControlUI)
+ run(): void
- playTurn(): void
- resetStaff(): void
- handleArrivals(): void
- buildWaiterOptions(): Map<MenuOption, Supplier<ActionResult>>
- waiterPhase(): void
- buildChefOptions(): Map<MenuOption, Supplier<ActionResult>>
- chefPhase(): void
- eatingAndPayment(): void
- endOfTurnDecay(): void
- waitingQueueLine(): List<String>
- occupiedTableLine(): List<String>
- customerStateLine(): List<String>
- averageOrNull(): Double
- checkVictory(): boolean

<<control>>
ActionResult

- success: boolean {readOnly}
- message: String {readOnly}

+ ActionResult(success: boolean, message: String)
+ isSuccess(): boolean
+ getMessage(): String

<<control>>
Restaurant

- waitingQueue: Queue<Customer> = new LinkedList<>() {readOnly}
- activeOrders: List<Order> = new ArrayList<>() {readOnly}
- tables: List<Table> = new ArrayList<>() {readOnly}
- menu: List<MenuItem> = new ArrayList<>() {readOnly}
- waiter: Waiter {readOnly}
- chef: Chef {readOnly}
- cashier: Cashier {readOnly}
- revenueCents: int = 0
- servedCount: int = 0
- unhappyDepartures: int = 0
- paidSatisfactions: List<Integer> = new ArrayList<>() {readOnly}

+ Restaurant(numTables: int)
+ getFreeTables(): List<Table>
+ getTables(): List<Table>
+ getMenuItems(): List<MenuItem>
+ getWaitingQueue(): Collection<Customer>
+ getSeatedWithoutOrder(): List<Customer>
+ getReadyOrders(): List<Order>
+ getOrdersinPrep(): List<Order>
+ getEatingCustomers(): List<Customer>
+ getReadytoPayCustomers(): List<Customer>
+ getCustomersLosingSatisfaction(): List<Customer>
+ getOrderFor(c: Customer): Order
+ getTableOf(c: Customer): Table
+ getRevenueCents(): int
+ getServedCount(): int
+ getUnhappyDepartures(): int
+ getPaidCount(): int
+ getAverageSatisfaction(): OptionalDouble
+ addToQueue(c: Customer): void
+ getWaiter(): Waiter
+ getCashier(): Cashier
+ getChef(): Chef
+ getAllStaff(): List<Staff>
+ seatCustomer(c: Customer, t: Table): ActionResult
+ takeOrder(c: Customer, rng: Random, currentTurn: int): ActionResult
+ advanceOrder(o: Order, currentTurn: int): ActionResult
+ serveOrder(o: Order, currentTurn: int): ActionResult
+ processPayment(c: Customer, currentTurn: int): String
+ processEndOfTurn(c: Customer): boolean

<<control>>
GameConfig

+ MAX_TURNS: int = 18 {readOnly}
+ NUM_TABLES: int = 2 {readOnly}
+ TARGET_PAID: int = 4 {readOnly}
+ TARGET_REVENUE: int = 4500 {readOnly}
+ TARGET_AVG_SATISFACTION: int = 60 {readOnly}
- TURNS: Integer[] = {1, 4, 7, 10, 13, 16} {readOnly}
- seed: long {readOnly}

+ GameConfig(seed: long)
+ returnCustomerThisTurn(turn: int): Customer
+ getSeed(): long

util

<<utility>>
MoneyFormatter
{final}

- MoneyFormatter()
+ format(amountInCents: int): String

<<create>>
<<use>>
ui
config
restaurant
cashier
waiter
chef
waitingQueue {ordered}
activeOrders {ordered}
tables {ordered}
menu {ordered}
inputReader
status
order / customer
table / customer
menuItem
0..*
0..1
1

Stage 3 Class Diagram

New role and menu policy - adds Host, ComboMeal and HappyHour

Exact source snapshot: main @ e6d2502 - all declared attributes and operations

+ public # protected - private ~ package/default underlined = static italic = abstract hollow/filled diamond = aggregation/composition hollow triangle = generalisation dashed arrow = selected creation/use dependency

default package

<<application>>
Main

+ Main() {implicit}
+ main(args: String[]): void

boundaries

<<boundary>>
ControlUI.MenuOption
{static, final}

- id: String {readOnly}
- label: String {readOnly}

+ MenuOption(id: String, label: String)
+ getId(): String
+ getLabel(): String
+ toString(): String

<<boundary>>
ControlUI
{final}

- DIVIDER: String {readOnly}
- inputReader: InputReader {readOnly}
- output: PrintStream {readOnly}

+ ControlUI()
+ ControlUI(inputReader: InputReader)
+ ControlUI(inputReader: InputReader, output: PrintStream)
+ showWelcome(seed: long): void
+ formatCustomerLabel(customerId: int, customerType: String): String
+ showTurnStart(turnNumber: int, maximumTurns: int, revenueCents: int
  averagePaidSatisfaction: Double, waitingQueue: List<String>,
  occupiedTables: List<String>, customerStates: List<String>): void
+ showTurnStart(turnNumber: int, maximumTurns: int, revenueCents: int
  averagePaidSatisfaction: Double, happyHourRemainingTurns: int
  waitingQueue: List<String>, occupiedTables: List<String>
  customerStates: List<String>): void
+ askHappyHourActivation(): boolean
+ askWaiterTask(availableOptions: List<MenuOption>): MenuOption
+ askChefTask(availableOptions: List<MenuOption>): MenuOption
+ askTask(staffName: String, availableOptions: List<MenuOption>): MenuOption
+ showArrival(customerDescription: String): void
+ showArrival(customerId: int, customerType: String): void
+ showOrderPlaced(customerDescription: String, menuItemName: String
  preparationUnits: int): void
+ showOrderPlaced(customerDescription: String, menuItemName: String, basePriceCents: int
  chargedPriceCents: int, preparationUnits: int): void
+ showCustomerBehavior(customerDescription: String, behaviourMessage: String): void
+ showPayment(customerDescription: String, amountPaidCents: int): void
+ showEvent(message: String): void
+ showAction(succeededMessage: String): void
+ showActionRejected(reason: String): void
+ showTurnResults(turnsUsed: int, servedCount: int, paidCount: int
  unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
+ showFinalResult(victory: boolean, turnsUsed: int, servedCount: int, paidCount: int
  unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
+ formatMoney(amountInCents: int): String
- showResults(heading: String, turnsUsed: int, servedCount: int, paidCount: int
  unhappyDepartures: int, revenueCents: int, averagePaidSatisfaction: Double): void
- printList(heading: String, items: List<String>, emptyMessage: String): void
- formatAverageSatisfaction(averagePaidSatisfaction: Double): String
- requireLine(stat: List<?>, name: String): void
- requireText(text: String, name: String): String

<<boundary>>
InputReader
{final}

- scanner: Scanner {readOnly}
- output: PrintStream {readOnly}

+ InputReader()
+ InputReader(scanner: Scanner, output: PrintStream)
+ readInt(prompt: String): int
+ readInt(prompt: String, minimum: int, maximum: int): int
+ readChoice(prompt: String, numberOfOptions: int): int
+ readLong(prompt: String): long
+ readYesNo(prompt: String): boolean
+ readNonBlank(prompt: String): String
+ readLine(prompt: String): String

control

<<control>>
TurnController

- config: GameConfig {readOnly}
- restaurant: Restaurant {readOnly}
- ui: ControlUI {readOnly}
- rng: Random {readOnly}
- happyHour: HappyHour = new HappyHour() {readOnly}
- currentTurn: int = 0

+ TurnController(config: GameConfig, ui: ControlUI)
+ run(): void
- playTurn(): void
- activateHappyHourAtTurnStart(): void
- recoverDuringHappyHour(): void
- hostPhase(): void
- resetStaff(): void
- handleArrivals(): void
- buildWaiterOptions(): Map<MenuOption, Supplier<ActionResult>>
- waiterPhase(): void
- buildChefOptions(): Map<MenuOption, Supplier<ActionResult>>
- chefPhase(): void
- eatingAndPayment(): void
- endOfTurnDecay(): void
- waitingQueueLine(): List<String>
- occupiedTableLine(): List<String>
- customerStateLine(): List<String>
- averageOrNull(): Double
- checkVictory(): boolean

<<control>>
ActionResult

- success: boolean {readOnly}
- message: String {readOnly}

+ ActionResult(success: boolean, message: String)
+ isSuccess(): boolean
+ getMessage(): String

util

<<utility>>
MoneyFormatter
{final}

- MoneyFormatter()
+ format(amountInCents: int): String

<<control>>
Restaurant

- waitingQueue: Queue<Customer> = new LinkedList<>() {readOnly}
- activeOrders: List<Order> = new ArrayList<>() {readOnly}
- tables: List<Table> = new ArrayList<>() {readOnly}
- menu: List<MenuItem> = new ArrayList<>() {readOnly}
- waiter: Waiter {readOnly}
- host: Host {readOnly}
- chef: Chef {readOnly}
- cashier: Cashier {readOnly}
- revenueCents: int = 0
- servedCount: int = 0
- unhappyDepartures: int = 0
- paidSatisfactions: List<Integer> = new ArrayList<>() {readOnly}

+ Restaurant(numTables: int)
+ getFirstTables(): List<Table>
+ getTables(): List<Table>
+ getMenuItems(): List<MenuItem>
+ getWaitingQueue(): Collection<Customer>
+ getSeatedWithoutOrder(): List<Customer>
+ getReadyOrders(): List<Order>
+ getOrdersInPrep(): List<Order>
+ getEatingCustomers(): List<Customer>
+ getReadyToPayCustomers(): List<Customer>
+ getCustomersLosingSatisfaction(): List<Customer>
+ getOrderFor(c: Customer): Order
+ getTableOf(c: Customer): Table
+ getRevenueCents(): int
+ getServedCount(): int
+ getUnhappyDepartures(): int
+ getPaidCount(): int
+ getAverageSatisfaction(): OptionalDouble
+ addToQueue(c: Customer): void
+ getWaiter(): Waiter
+ getCashier(): Cashier
+ getChef(): Chef
+ getHost(): Host
+ getAllStaff(): List<Staff>
+ seatCustomer(c: Customer, t: Table): ActionResult
+ hostSeatCustomer(c: Customer, t: Table): ActionResult
+ takeOrder(c: Customer, mg: Random, currentTurn: int, happyHour: HappyHour): ActionResult
+ takeOrder(c: Customer, mg: Random, currentTurn: int): ActionResult
+ advanceOrder(o: Order, currentTurn: int): ActionResult
+ serveOrder(o: Order, currentTurn: int): ActionResult
+ processPayment(c: Customer, currentTurn: int): String
+ processEndOfTurn(c: Customer): boolean
+ recoverWaitingCustomers(points: int): void

<<control>>
GameConfig

+ MAX_TURNS: int = 18 {readOnly}
+ NUM_TABLES: int = 2 {readOnly}
+ TARGET_PAID: int = 4 {readOnly}
+ TARGET_REVENUE: int = 4500 {readOnly}
+ TARGET_AVG_SATISFACTION: int = 60 {readOnly}
+ HAPPY_HOUR_RECOVERY: int = 15 {readOnly}
- TURNS: Integer[] = {1, 4, 7, 10, 13, 16} {readOnly}
- seed: long {readOnly}

+ GameConfig(seed: long)
+ returnCustomerInTurn(turn: int): Customer
+ getSeed(): long

<<control>>
HappyHour
{final}

+ RECOVERY_POINTS: int = 15 {readOnly}
- activationTurn: int = -1

+ HappyHour() {implicit}
+ canActivate(): boolean
+ activate(turn: int): void
+ isActive(turn: int): boolean
+ getRemainingTurns(turn: int): int
+ priceForOrder(priceAfterCustomerDiscountCents: int, turn: int): int

entities

<<entity>>
Staff
{abstract}

- name: String
- actedThisTurn: boolean = false

+ Staff(name: String)
+ getName(): String
+ canAct(): boolean
+ markActed(): void
+ resetForNewTurn(): void

<<entity>>
Chef
{final}

+ Chef()
+ Chef(name: String)
+ prepareOneUnit(order: Order, turn: int): boolean

<<entity>>
Cashier
{final}

+ Cashier()
+ Cashier(name: String)
+ collectPayment(customer: Customer, turn: int): int

<<entity>>
Waiter
{final}

+ Waiter()
+ Waiter(name: String)
+ seatCustomer(customer: Customer, table: Table): void
+ takeOrder(customer: Customer, item: MenuItem, orderId: int, turn: int): Order
+ takeOrder(customer: Customer, item: MenuItem, turn: int): Order
+ serve(customer: Customer, turn: int): void

<<entity>>
Host
{final}

+ Host()
+ Host(name: String)
+ seatCustomer(customer: Customer, table: Table): void

<<entity>>
Customer
{abstract}

- nextId: int = 1
- customerId: int {readOnly}
- arrivalTurn: int {readOnly}
- satisfaction: int
- status: CustomerStatus
- order: Order
- table: Table
- servedTurn: int
- readyToPayTurn: int
- satisfactionAtPayment: int

Customer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getPriceCents(menuItem: MenuItem): int
+ applyEndOfTurn(): boolean
+ increaseSatisfaction(points: int): void
+ decreaseSatisfaction(): void
+ abandon(): void
+ markSeated(assignedTable: Table): void
+ chooseMenuItem(menu: List<MenuItem>, rng: Random): MenuItem
+ attachOrder(newOrder: Order): void
+ markServed(turn: int): void
+ finishEating(currentTurn: int): boolean
+ markPaid(currentTurn: int): int
+ releaseTable(table: Table): void
+ getCustomerId(): int
+ getArrivalTurn(): int
+ getSatisfaction(): int
+ getStatus(): CustomerStatus
+ getOrder(): Order
+ getTable(): Table
+ getServedTurn(): int
+ getReadyToPayTurn(): int
+ getSatisfactionAtPayment(): int
+ getTypeName(): String

<<entity>>
CriticCustomer
{final}

- SATISFACTION_LOSS_PER_TURN: int = 12 {readOnly}

+ CriticCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getTypeName(): String
+ chooseMenuItem(menu: List<MenuItem>, rng: Random): MenuItem

<<entity>>
RegularCustomer
{final}

- SATISFACTION_LOSS_PER_TURN: int = 8 {readOnly}

+ RegularCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getTypeName(): String

<<entity>>
VIPCustomer
{final}

- SATISFACTION_LOSS_PER_TURN: int = 5 {readOnly}

+ VIPCustomer(arrivalTurn: int)
+ getSatisfactionLoss(): int
+ getPriceCents(menuItem: MenuItem): int
+ getTypeName(): String

<<entity>>
Table
{final}

- tableNumber: int {readOnly}
- customer: Customer

+ Table(tableNumber: int)
+ getTableNumber(): int
+ getCustomer(): Customer
+ isOccupied(): boolean
+ seatCustomer(customer: Customer): void
+ freeTable(): void
+ toString(): String

<<entity>>
Order
{final}

- generatedIds: int = 1
- orderId: int {readOnly}
- customer: Customer {readOnly}
- menuItem: MenuItem {readOnly}
- priceCents: int {readOnly}
- requiredPreparationUnits: int {readOnly}
- createdTurn: int {readOnly}
- completedPreparationUnits: int
- readyTurn: int = -1
- servedTurn: int = -1
- paidTurn: int = -1
- status: OrderStatus

+ Order(orderId: int, customer: Customer, menuItem: MenuItem, createdTurn: int)
+ Order(customer: Customer, menuItem: MenuItem, createdTurn: int)
+ Order(customer: Customer, menuItem: MenuItem, createdTurn: int, chargedPriceCents: int)
+ Order(orderId: int, customer: Customer, menuItem: MenuItem, createdTurn: int
  chargedPriceCents: int)
- nextGeneratedId(): int
+ advancePreparation(turn: int): boolean
~ markServed(turn: int): void
~ markPaid(turn: int): void
+ cancel(): void
+ getOrderId(): int
+ getCustomer(): Customer
+ getMenuItem(): MenuItem
+ getPriceCents(): int
+ getRequiredPreparationUnits(): int
+ getCompletedPreparationUnits(): int
+ getCreatedTurn(): int
+ getReadyTurn(): int
+ getServedTurn(): int
+ getPaidTurn(): int
+ getStatus(): OrderStatus

<<enumeration>>
CustomerStatus

WAITING
SEATED
SERVED
READY_TO_PAY
PAID
LEFT

<<enumeration>>
OrderStatus

ORDERED
READY
SERVED
PAID
CANCELLED

<<entity>>
MenuItem

+ SALAD: MenuItem = new MenuItem("Salad", 900, 1) {readOnly}
+ BURGER: MenuItem = new MenuItem("Burger", 1500, 1) {readOnly}
+ PASTA: MenuItem = new MenuItem("Pasta", 1800, 2) {readOnly}
- STAGE_ONE_MENU: List<MenuItem> = Arrays.asList(SALAD, BURGER, PASTA) {readOnly}
- STAGE_THREE_MENU: List<MenuItem> {readOnly}
- name: String {readOnly}
- priceCents: int {readOnly}
- preparationUnits: int {readOnly}

+ MenuItem(name: String, priceCents: int, preparationUnits: int)
+ stageOneMenu(): List<MenuItem>
+ stageThreeMenu(): List<MenuItem>
+ getName(): String
+ getPriceCents(): int
+ getPrice(): int
+ getPreparationUnits(): int
+ getPrepUnits(): int
+ toString(): String

<<entity>>
ComboMeal
{final}

- first: MenuItem {readOnly}
- second: MenuItem {readOnly}

+ ComboMeal(first: MenuItem, second: MenuItem)
- component(item: MenuItem): MenuItem
- nameOf(first: MenuItem, second: MenuItem): String
- priceOf(first: MenuItem, second: MenuItem): int
- prepUnitsOf(first: MenuItem, second: MenuItem): int
+ getFirst(): MenuItem
+ getSecond(): MenuItem

<<create>>
<<use>>
<<read>>
config
happyHour
restaurant
ui
chef
cashier
tables {ordered}
waiter
host
activeOrders {ordered}
waitingQueue {ordered}
inputReader
status
table / customer
order / customer
menuItem
components {first, second}
menu {ordered}